

Resource Allocation System

Iteration 31 – Extended User and Organization Model with Permissions and Priorities

Najib Attar

May 2026

1 Iteration Goal

The goal of Iteration 31 is to extend the user and organization model so that the system better represents a university environment. Earlier iterations already introduced a polymorphic user hierarchy with `Student`, `Instructor`, `Staff`, and `Administrator`. This iteration preserves that working structure and adds new domain concepts around users, roles, and organization units.

The existing `User` class remains the system-facing base class. It was not renamed to `Person`, because many backend components already depend on `User`, including requests, factories, rules, and allocation strategies.

2 Changes from Previous Iteration

Iteration 30 improved the scheduling model by adding minute-based time slot support. Iteration 31 focuses on the user and organization domain model.

The main changes are:

- Added `TeachingAssistant` as a new `User` subclass.
- Added `Role` as a structural model for future role-based extensions.
- Added `OrganizationUnit` as a base organization class.
- Added `Department`, `Faculty`, `Club`, `AdministrativeOffice`, and `ResearchGroup`.
- Extended `User` with optional profile fields: `email`, `status`, `primaryUnitName`, and `assignedRoleNames`.
- Added optional subclass-specific fields: `studentNo`, `program`, `yearLevel`, `title`, `officeRoom`, `assistantType`, `jobTitle`, and `adminLevel`.
- Updated `UserFactory` to create `TeachingAssistant` users.

- Updated user loading so old and extended `users.csv` rows remain compatible.
- Added a `TeachingAssistant` example to `users.csv`.

3 Scenario

A university resource allocation system may involve more than students, instructors, staff, and administrators. Teaching assistants may support teaching activities, reserve rooms for lab support, or hold office hours. However, official exam requests should still be limited to instructors and administrators.

For this reason, `TeachingAssistant` is added as a separate user type. It can request classrooms, laboratories, and meeting rooms, but it cannot submit exam requests. This allows the model to represent teaching assistants more accurately without changing the existing rule engine or allocation strategies.

The organization model is also extended. Departments, faculties, clubs, administrative offices, and research groups are represented as organization units. These classes prepare the system for future unit-based reporting or permission extensions, but they do not replace the current permission system in this iteration.

4 Modules and Their Tasks

Module	Task in Iteration 31
User	Remains the system-facing base class and stores optional profile fields such as email, status, primary unit name, and assigned role names.
Student	Keeps existing student behavior and stores optional student-specific data such as student number, program, and year level.
TeachingAssistant	Adds a new user subtype with its own priority and permission behavior.
Staff	Keeps staff behavior and stores optional job title information.
Instructor	Keeps instructor behavior and stores optional title and office room information.
Administrator	Keeps administrator behavior and stores optional administrator level information.
UserFactory	Creates <code>Student</code> , <code>TeachingAssistant</code> , <code>Staff</code> , <code>Instructor</code> , and <code>Administrator</code> objects.
DataLoader	Loads both old-format and extended user rows safely from <code>users.csv</code> .
Role	Provides a structural role model with role ID, role name, and permissions for future use.

<code>OrganizationUnit</code>	Provides a base class for university organization units.
<code>Department,</code>	Represent specific types of university organization units.
<code>Faculty,</code>	
<code>Club,</code>	
<code>AdministrativeOffice,</code>	
<code>ResearchGroup</code>	
<code>UserRoleRule</code>	Continues checking whether the user can request the selected space type.
<code>RequestTypeRule</code>	Continues checking whether the user can submit the selected request type.
<code>PriorityAllocationStrategy</code>	Continues using <code>getPriority()</code> to order requests by user priority.

5 Extended User and Organization Model

The existing user hierarchy is preserved. `User` remains the abstract base class, and all user subclasses still implement the same behavior methods:

- `getRoleName()`
- `getPriority()`
- `canRequestSpaceType(...)`
- `canSubmitRequestType(...)`

This keeps the rule engine compatible with the previous design. The new `TeachingAssistant` class follows the same polymorphic structure as the existing user types.

The organization model is added separately. `OrganizationUnit` is the base class for `Department`, `Faculty`, `Club`, `AdministrativeOffice`, and `ResearchGroup`. These classes are structural domain additions. They prepare the system for future extensions but do not own allocation rules in this iteration.

The `Role` class is also structural. It stores role identity and permissions, but `UserRoleRule` and `RequestTypeRule` still rely on polymorphic user behavior. This avoids a risky redesign and keeps the current system stable.

6 User Permission and Priority Matrix

Higher priority values represent higher scheduling priority. The `PriorityAllocationStrategy` uses these values when ordering requests.

User Type	Priority
Student	1
TeachingAssistant	2
Staff	3
Instructor	4
Administrator	5

Table 2: Priority values used by the priority allocation strategy.

User Type	Classroom	Laboratory	MeetingRoom
Student	Yes	No	Yes
TeachingAssistant	Yes	Yes	Yes
Staff	No	No	Yes
Instructor	Yes	Yes	Yes
Administrator	Yes	Yes	Yes

Table 3: Space-type permissions checked conceptually by `UserRoleRule`.

`UserRoleRule` checks whether a user is allowed to request or use the requested space type. For example, a student may request a classroom or meeting room, but not a laboratory.

User Type	OneTime	Recurring	Exam
Student	Yes	Yes	No
TeachingAssistant	Yes	Yes	No
Staff	Yes	Yes	No
Instructor	Yes	Yes	Yes
Administrator	Yes	Yes	Yes

Table 4: Request-type permissions checked conceptually by `RequestTypeRule`.

`RequestTypeRule` checks whether a user is allowed to submit the selected request type. Exam requests are limited to instructors and administrators. `InvalidRequest` is not shown as a real user-submitted request type because it is created internally to preserve malformed input rows safely.

`UserRoleRule` and `RequestTypeRule` remain separate. `UserRoleRule` controls space-type authorization, while `RequestTypeRule` controls request-type authorization. `PriorityAllocationStrategy` uses the priority value separately. This separation prevents mixing different responsibilities inside one rule.

7 Iteration Comparison

Iter.	Focus	Added / Modified	Prototype Effect
26	Request-type permission	RequestTypeRule	Separated request-type authorization from space-type authorization.
27	Allocation preparation	assignedParticipants and space pool access	Prepared allocations and strategies for multi-room exam allocation.
28	Multi-room greedy	MultiRoomExamGreedyStrategy	Allowed splittable exams to use multiple classrooms.
29	Multi-room best-fit	MultiRoomExamBestFitStrategy	Reduced unused capacity for splittable exam allocations.
30	Minute-based time slots	Minute-based TimeSlot	Improved scheduling precision and overlap detection.
31	Extended user model	TeachingAssistant , Role , and organization units	Expanded the university domain model while preserving existing allocation behavior.

8 Updated Class Diagram

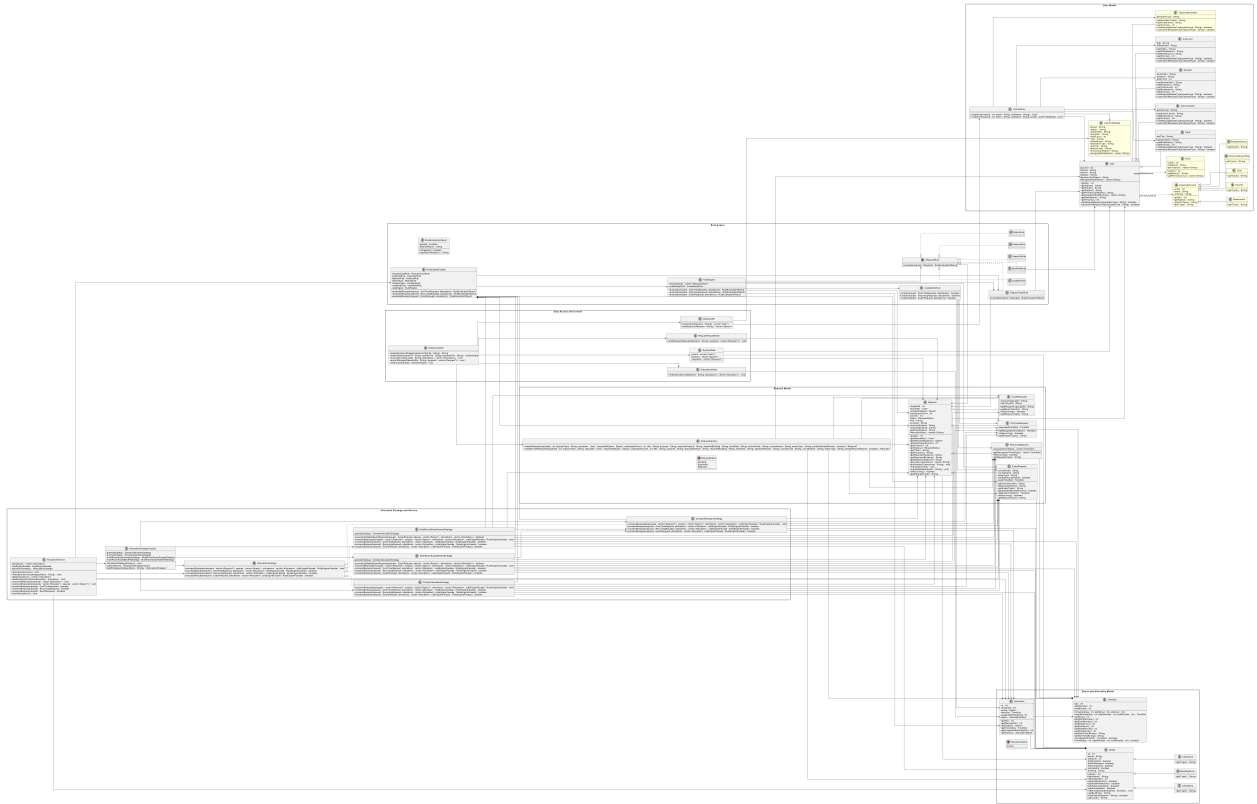


Figure 1: Updated class diagram after extending the user and organization model.

The updated class diagram includes **TeachingAssistant** as a subclass of **User**. It also includes **Role** and the **OrganizationUnit** hierarchy with **Department**, **Faculty**, **Club**, **AdministrativeOffice**, and **ResearchGroup**.

9 Updated Workflow Diagram

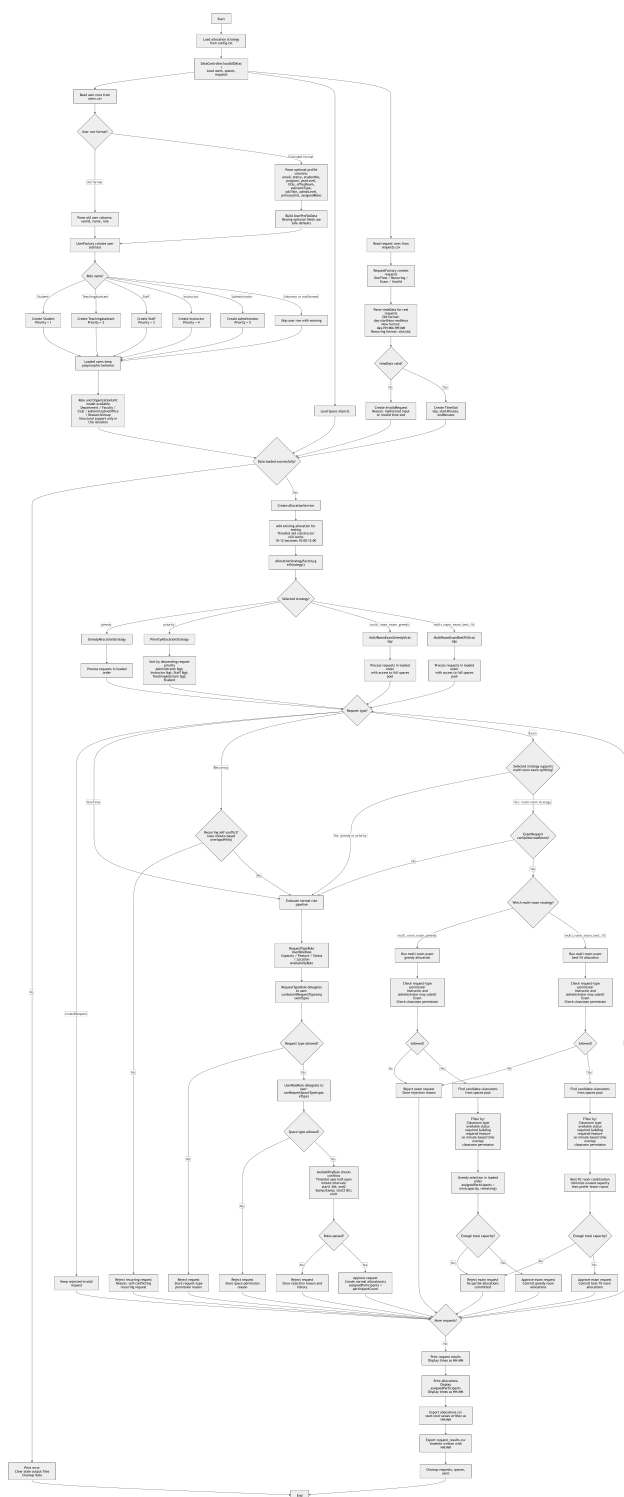


Figure 2: Updated workflow showing user loading, factory creation, and rule usage.

The workflow is mostly unchanged. `DataLoader` loads user rows from `users.csv`. `UserFactory` creates the correct user subtype, including `TeachingAssistant`. Later, `UserRoleRule` and `RequestTypeRule` use polymorphic permission behavior, while `PriorityAllocationStrategy` uses `getPriority()` when priority-based processing is selected.

10 Prototype Implementation

The prototype now supports the following user types:

- `Student`
- `TeachingAssistant`
- `Staff`
- `Instructor`
- `Administrator`

A teaching assistant can submit one-time and recurring requests and can request classrooms, laboratories, and meeting rooms. However, teaching assistants cannot submit exam requests. This keeps official exam scheduling limited to instructors and administrators.

The `users.csv` loading remains backward-compatible. Existing rows with only `userId`, `name`, and `role` still load correctly. Extended user fields are optional and use safe defaults when missing.

Testing confirmed that existing user types still load and behave correctly. It also confirmed that `TeachingAssistant` loads through `UserFactory`, can submit normal requests, and is rejected from submitting exam requests by `RequestTypeRule`.

11 Summary

Iteration 31 extends the user and organization domain model while preserving the existing allocation system. The implementation adds `TeachingAssistant`, `Role`, and the `OrganizationUnit` hierarchy without replacing the current permission system.

The key design decision is to keep `User` as the system-facing base class. Permission behavior remains inside user subclasses, and the existing rules continue to delegate to polymorphic user methods. This keeps the backend stable while preparing the model for richer university structures in future iterations.